MIS41110 Practical Week 1, Solutions

Covers the Week 1 lecture (types, math, strings, variables) · **Worked** Monday of Week 2

Exercise 0: the setup gate

The bug is a missing closing quote, so Python never sees the end of the string:

```
# Week-1 sheet, exercise 0 -- the setup gate. Steps 1 and 2 leave no file to keep:
# step 1 is the REPL check (python3, >>> appears, quit()), step 2 is creating and
# running hello_farpoint.py from its own folder. This file is step 3's fixed line.
print("Radio check: all stations, please respond")
```

Python prints `SyntaxError: unterminated string literal` and puts a caret under the opening quote. The line it names is wherever the student pasted it: line 2 if it went under step 2's working line, as the sheet asks, and line 1 in a file of its own. Reading the message rather than guessing is the habit worth starting today: it names both the problem and the line.

Exercise 1: supply-crate arithmetic

No f-strings yet (those arrive in the week-2 lecture), so the receipt is glued with `str()` and `+`:

```
# Week-1 sheet, exercise 1 -- supply-crate arithmetic (no f-strings yet: str() + "+").
a = 17
b = 5

print("Value a is", type(a))     # commas: print takes several values (taught)
print("Value b is", type(b))
print(str(a) + " + " + str(b) + " = " + str(a + b))
print(str(a) + " - " + str(b) + " = " + str(a - b))
print(str(a) + " * " + str(b) + " = " + str(a * b))
print(str(a) + " / " + str(b) + " = " + str(a / b))    # (b) this one: / always gives a
    ↪ float
print(str(a) + " % " + str(b) + " = " + str(a % b))
print(str(a) + " ** " + str(b) + " = " + str(a ** b))
```

(a) Because every printed value is computed from `a` and `b` rather than typed, changing the two assignments changes all eight lines.

(b) `17 / 5 = 3.4`. True division always returns a `float`, even when the division is exact; the other five arithmetic lines stay `int` (the first two lines print `type` objects, not values).

Exercise 2: the station door sign

One `print`, one string, escapes for the quotes and the newlines:

```
# Week-1 sheet, exercise 2 -- the station door sign: one print, escapes, string math.
print("=" * 30 + "\n   FARPOINT STATION\n   \"South of everywhere\"\n   Crew: 44
    ↪ Winter: yes\n" + "=" * 30)
```

(a) `"=" * 30` is string multiplication, the same operator doing a different job because the left operand is a string.

Exercise 3: temperature math

```
# Week-1 sheet, exercise 3 -- temperature math.
t1 = -17.1
t2 = -17.1
t3 = -17.1

print(t1 + t2 + t3)
# Not exactly -51.3: floats are stored in binary, and 0.1 (so 17.1 too) has no exact
# binary form, so tiny representation errors add up -- the week-1 wall.
print((t1 + t2 + t3) / 3)
print((9 / 5) * t1 + 32)
```

The sum prints `-51.300000000000004`, not `-51.3`. That is the week-1 wall: floats are stored in binary, `17.1` has no exact binary form, and three inexact values added together show the error.

Exercise 4: predict, then run

```
# Week-1 sheet, exercise 4 -- predict, then run.
print(2 + 3 * 4)        # 14  (* before +)
print((2 + 3) * 4)      # 20  (parentheses first)
print(3 * 2 ** 2)       # 12  (** binds tightest, then *)
print(7 // 2)           # 3
print(7 % 2)            # 1
print(True + True)      # 2   (True behaves as 1)
print(str(3) + str(0.5))  # 30.5   <- a STRING (a)
print(int("13") + 5)    # 18
print("13" + str(5))    # 135    <- a STRING (a)
```

(a) `str(3) + str(0.5)` gives `"30.5"` and `"13" + str(5)` gives `"135"`. Both are string concatenation that happens to look numeric, which is the trap.

Meaningful variable names, a comment where a line needs one, and run your file after every change.